

App de Notas Markdown

Guía paso a paso — FastAPI + React

JWT · SQLite → PostgreSQL · Docker

Mapa de implementación — sin código, un concepto a la vez

Contexto

API de notas en Markdown con FastAPI, autenticación JWT, CRUD de notas por usuario, frontend React con renderizado de Markdown sanitizado, y preparación para despliegue con Docker. Base de datos: SQLite primero, PostgreSQL al final. Deben cumplirse los criterios de aceptación CA 1.1–1.3 (auth) y CA 2.1–2.3 (notas).

Regla de oro: no avances de fase sin pasar su verificación. Si una fase satura, párala en su bloque (3a, 3b, 3c...) — cada bloque es un cierre válido del día.

Total estimado: 2–3 semanas a ritmo de bloques cortos.

Fase 0 — Preparar el terreno medio día

1. Crea la carpeta del proyecto con dos subcarpetas: backend/ y frontend/.
2. En backend/: crea un entorno virtual de Python y actívalo.
3. Instala solo lo mínimo para arrancar: fastapi y uvicorn. Lo demás se instala cuando se necesite.
4. Inicializa git en la raíz del proyecto y haz el primer commit con un .gitignore (entorno virtual, __pycache__, .env, node_modules).

Verificación: git status limpio, entorno virtual activo, pip list muestra fastapi.

Docs: [venv \(entornos virtuales\)](#) · [FastAPI](#) · [.gitignore](#)

Fase 1 — API "hola mundo" con documentación automática medio día

1. Crea backend/app/main.py con una app FastAPI y un endpoint GET /health que devuelva un JSON simple.
2. Arranca el servidor con uvicorn en modo recarga automática.
3. Abre /docs en el navegador — Swagger autogenerado, tu tablero de pruebas durante todo el proyecto.

Concepto: qué es un endpoint, qué hace uvicorn, qué es Swagger.

Verificación: /health responde 200 en el navegador y aparece en /docs.

Docs: [FastAPI — primeros pasos](#) · [Uvicorn](#) · [Swagger/OpenAPI en FastAPI](#)

Fase 2 — Base de datos y modelos

1–2 días

1. Instala sqlalchemy.
2. Crea la estructura de capas desde ahora (aunque casi vacía): `app/routers/`, `app/services/`, `app/repositories/`, `app/models/`, `app/schemas/`.
3. Configura la conexión a SQLite (archivo local). Cambiar a PostgreSQL después será cambiar una línea de configuración.
4. Define dos tablas: **users** (id, email único, contraseña hashada, fecha de creación) y **notes** (id, título, contenido markdown crudo, fecha de creación, fecha de actualización, FK al usuario dueño).
5. Haz que las tablas se creen al arrancar la app.

Concepto: ORM, tablas como clases, relación uno-a-muchos (un usuario tiene muchas notas).

Verificación: al arrancar la app se crea el archivo `.db`. Ábrelo con la extensión SQLite de VS Code (o `sqlite3` en terminal) y confirma las dos tablas con sus columnas.

Docs: [SQLAlchemy ORM quickstart](#) · [FastAPI + SQL databases](#)

Fase 3 — Registro y login con JWT

2–3 días, divídela

Bloque 3a — Registro (CA 1.1)

1. Instala `passlib[bcrypt]` (o `bcrypt` directo) y `pydantic[email]`.
2. Schema Pydantic de registro: email + contraseña, con validación de formato de email.
3. POST `/auth/register`: recibe email y contraseña, hashada con `bcrypt` (nunca texto plano), guarda el usuario. Email ya existe → 409.
4. Sigue las capas: router → servicio (hashada) → repositorio (toca la BD).

Verificación: registras un usuario desde `/docs`; en la BD la contraseña es un hash que empieza con `$2b$`. Registrar el mismo email dos veces da 409.

Docs: [Passlib — bcrypt](#) · [Pydantic — validación \(EmailStr\)](#)

Bloque 3b — Login y emisión de token (CA 1.2)

1. Instala una librería JWT para Python (`python-jose` o `pyjwt`).
2. Crea `.env` con la clave secreta del JWT y el tiempo de expiración. Instala `python-dotenv` o usa Pydantic Settings. La clave secreta nunca va al código ni a git.
3. POST `/auth/login`: busca el usuario por email, verifica contraseña contra el hash con `bcrypt`; si coincide devuelve un JWT con el id del usuario y fecha de expiración.
4. Email no existe o contraseña no coincide → 401 (mismo mensaje en ambos casos, para no revelar cuál falló).

Verificación: login desde /docs devuelve un token. Pégallo en jwt.io y confirma que dentro va el id del usuario y el campo exp.

Docs: [PyJWT](#) · [python-jose](#) · [Pydantic Settings \(.env\)](#) · [jwt.io \(debugger\)](#)

Bloque 3c — Proteger endpoints (CA 1.3)

1. Dependencia de FastAPI `get_current_user`: lee `Authorization: Bearer <token>`, valida firma y expiración, busca el usuario y lo devuelve. Falla → 401.
2. Endpoint de prueba GET `/auth/me` protegido con esa dependencia, devuelve el email del usuario logueado.

Concepto: hash vs encriptación, qué es un JWT (encabezado.datos.firma), qué es una dependencia en FastAPI.

Verificación: `/auth/me` sin token → 401. Con token válido → tus datos. Con token expirado o alterado → 401. En /docs usa el botón "Authorize".

Docs: [FastAPI — OAuth2 con JWT](#) · [FastAPI — dependencias](#)

Fase 4 — CRUD de notas 2-3 días

Todos estos endpoints usan la dependencia de la fase 3c — sin token válido, 401 automático.

1. **CA 2.1 — Crear:** POST `/notes` recibe title y content (markdown), validados con Pydantic (título no vacío, límites de longitud razonables). Se guarda asociada al usuario del token. Devuelve 201 con la nota creada.
2. **CA 2.2 — Listar:** GET `/notes` devuelve solo las notas del usuario autenticado, paginadas (page/size o skip/limit), y solo metadatos: id, título, fecha de actualización. Sin el contenido.
3. **CA 2.3 — Ver una:** GET `/notes/{id}` devuelve la nota completa. Existe y es tuya → 200; existe pero es de otro usuario → 403; no existe → 404.
4. **Editar:** PUT `/notes/{id}` (o PATCH) actualiza título/contenido y refresca la fecha de actualización. Mismas reglas 403/404.
5. **Borrar:** DELETE `/notes/{id}` → 204. Mismas reglas 403/404.

Decisión ya tomada: el backend guarda y devuelve markdown crudo — no lo convierte a HTML. El frontend se encarga de renderizar.

Verificación (prueba de fuego): crea DOS usuarios. Con A crea 3 notas. Confirma que B no las ve en su lista, y si B pide una nota de A por id recibe 403. Prueba la paginación creando más notas de las que caben en una página.

Ponytail check: deja un `test_notes.py` pequeño (pytest + TestClient) que cubra al menos: crear nota, listar, y el caso 403 entre usuarios.

Docs: [FastAPI — query params \(paginación\)](#) · [FastAPI — manejo de errores \(403/404\)](#) · [FastAPI — testing con TestClient](#) · [pytest](#)

Fase 5 — Frontend React

3–5 días, en bloques

Bloque 5a — Proyecto y primera pantalla

1. En frontend/: crea el proyecto con Vite (plantilla React). Instala Tailwind CSS siguiendo su guía oficial para Vite.
2. Configura CORS en el backend (middleware de FastAPI) permitiendo el origen del dev server de Vite.
3. Haz que React llame a GET /health y muestre el resultado — prueba que las dos mitades se hablan.

Verificación: la página muestra la respuesta del backend sin errores CORS en la consola del navegador.

Docs: [Vite — guía](#) · [Tailwind CSS + Vite](#) · [FastAPI — CORS](#)

Bloque 5b — Login y registro

1. Pantallas de registro y login (formularios simples con Tailwind).
2. Al hacer login, guarda el token JWT (localStorage es aceptable para esta entrega) y el estado "logueado" en React.
3. Función central de fetch que agregue Authorization: Bearer <token> a toda petición, y que al recibir 401 borre el token y mande al login.
4. Botón de cerrar sesión = borrar token.

Verificación: registro → login → recargas la página y sigues logueado → logout → intentar ver notas te manda al login.

Docs: [React — referencia](#) · [MDN — localStorage](#) · [MDN — Fetch API](#)

Bloque 5c — Lista y CRUD de notas

1. Pantalla principal: lista de notas del usuario (título + fecha), con paginación (botones anterior/siguiente bastan).
2. Crear nota, abrir nota, editar, borrar — conectando cada acción a su endpoint.

Verificación: el ciclo completo crear → ver en lista → abrir → editar → borrar funciona desde la interfaz.

Bloque 5d — Renderizado de Markdown seguro (requisito de seguridad clave)

1. Instala react-markdown (por defecto no renderiza HTML incrustado, que es justo lo seguro) y renderiza el contenido de la nota.
2. Vista del editor: textarea con el markdown a la izquierda, vista previa en vivo a la derecha.
3. **Prueba XSS obligatoria:** crea una nota cuyo contenido incluya una etiqueta <script> y una imagen con onerror. Nada debe ejecutarse al ver la nota. Si en algún punto habilitas HTML crudo en react-

markdown, ahí DOMPurify se vuelve obligatorio — documenta en el README cuál de las dos rutas tomaste.

Verificación: encabezados, listas, negritas y código se ven bien; el intento de XSS no ejecuta nada.

Docs: [react-markdown](#) · [DOMPurify](#)

Fase 6 — Migrar a PostgreSQL 1 día

1. Levanta PostgreSQL con Docker (un contenedor, sin instalar nada en Windows).
2. Instala el driver de PostgreSQL para Python (psycopg2-binary o psycopg).
3. Mueve la URL de conexión al `.env` y cámbiala de SQLite a PostgreSQL. Si SQLAlchemy quedó bien montado, no cambia nada más.
4. Arranca la app: las tablas se crean en PostgreSQL. Vuelve a correr `test_notes.py`.

Verificación: los tests pasan igual que con SQLite y puedes registrarte/crear notas contra PostgreSQL.

Docs: [Imagen oficial PostgreSQL en Docker Hub](#) · [psycopg2](#)

Fase 7 — Docker y entrega 1-2 días

1. Dockerfile del backend: imagen de Python, instala dependencias desde `requirements.txt` (genéralo ahora), copia el código, arranca uvicorn.
2. docker-compose con dos servicios: backend + PostgreSQL, con variables de entorno (secreto JWT, URL de BD) inyectadas desde el compose — nunca horneadas en la imagen.
3. Frontend: `npm run build` genera los archivos estáticos. Para la entrega basta documentar cómo servirlos (o añadir un tercer servicio nginx al compose para el punto extra).
4. README con: cómo levantar todo (`docker compose up`), cómo correr los tests, las decisiones tomadas (markdown se renderiza en el cliente, estrategia de sanitización, SQLite→PostgreSQL) — los evaluadores leen esto primero.

Docs: [Referencia de Dockerfile](#) · [Docker Compose](#) · [nginx \(Docker Hub\)](#)

Verificación final — checklist de criterios de aceptación

- ☐ CA 1.1: registro con email + contraseña hasheada con bcrypt
- ☐ CA 1.2: login devuelve JWT con expiración
- ☐ CA 1.3: endpoints de notas sin token → 401
- ☐ CA 2.1: POST /notes valida con Pydantic y asocia al usuario
- ☐ CA 2.2: GET /notes paginado, solo metadatos, solo del usuario

- ☐ CA 2.3: GET /notes/{id} completo; nota ajena → 403
- ☐ Editar y borrar notas funcionan con reglas 403/404
- ☐ Markdown renderizado en el cliente, prueba XSS no ejecuta código
- ☐ Capas separadas: routers / services / repositories
- ☐ docker compose up levanta backend + PostgreSQL
- ☐ Tests mínimos pasan

Recortado a propósito (no lo piden los criterios de aceptación): refresh tokens, roles/permisos, búsqueda de notas, Alembic para migraciones (crear tablas al arrancar basta para la entrega), despliegue serverless (Docker cumple el requisito de "preparado para producción").